

Locational Break-Even Analysis Without Plotting

A lower-envelope algorithm and an open-source bilingual teaching tool

Ramiz Assaf

Department of Industrial Engineering, An-Najah National University, Nablus, Palestine

White paper, version 1.0, October 2026

Software: <https://github.com/ramizassaf/break-even-location-finder>

Abstract

Locational break-even analysis selects the alternative with the lowest total cost, $TC = FC + VC \times Q$, for each level of annual volume Q . Operations management textbooks teach the method through a hand-drawn graph of total cost lines (Heizer, Render, & Munson, 2020). The graph works for three or four alternatives, but the reading of crossing points depends on drawing accuracy, and the method gives students no rule for deciding which crossings matter. This paper states the problem as the computation of the lower envelope of a set of lines, a classic problem in computational geometry (de Berg, Cheong, van Kreveld, & Overmars, 2008). We present a stack algorithm with $O(n \log n)$ running time that returns the winning alternatives in order of volume and the break-even quantities between them, with no graph. We give a short correctness argument, a dual interpretation through the convex hull of the points (VC, FC) , a worked example, and a verification against brute force on 5,000 random instances. The algorithm ships as an open-source web application with English and Arabic interfaces, a Python command line tool, and a test suite.

1. Introduction

A firm choosing a plant location, a process, or a make-or-buy option often faces alternatives with a trade-off: one option has a low fixed cost and a high variable cost, another has the reverse. For each alternative i , annual total cost is

$$TC_i(Q) = FC_i + VC_i \times Q$$

where FC_i is the annual fixed cost, VC_i is the variable cost per unit, and Q is annual volume. The decision maker wants the cheapest alternative for each Q and the volumes where the preferred alternative changes.

Textbooks call this locational cost-volume analysis or locational break-even analysis and solve the problem in three steps: compute TC for each site, plot the lines on one graph, and read the volume ranges where each site has the lowest line (Heizer et al., 2020, Chapter 8). The same tool appears under the name crossover chart in process selection.

The graphical method has three weaknesses in teaching and in practice:

1. **Accuracy.** Students read crossing points from a sketch. Close crossings produce wrong ranges.
2. **No selection rule.** With n alternatives, up to $n(n - 1)/2$ pairs cross. Only some crossings define ranges. The graph gives no rule for choosing the relevant ones without looking.
3. **Scale.** Beyond five or six alternatives, the graph becomes unreadable.

This paper answers a question students ask in class: how do you know which lines to examine without plotting? Section 2 states the problem. Section 3 reviews related work in operations management, linear programming, and computational geometry. Section 4 presents the algorithm and a correctness argument. Section 5 gives the convex hull view. Section 6 works an example. Section 7 reports verification. Section 8 describes the software, and Section 9 discusses classroom use and extensions.

2. Problem statement

Input. A set of n alternatives, each with $FC_i \geq 0$ and $VC_i \geq 0$.

Output.

1. The ordered list of alternatives that are cheapest over some interval of $Q \geq 0$, from low volume to high volume.
2. The break-even quantities $Q_1^* < Q_2^* < \dots$ between consecutive winners.
3. The alternatives that are never cheapest.

The break-even quantity between alternatives i and j , with $VC_i > VC_j$, solves $TC_i(Q) = TC_j(Q)$:

$$Q^*(i, j) = (FC_j - FC_i) / (VC_i - VC_j)$$

Define the minimum cost function

$$f(Q) = \min_i [FC_i + VC_i \times Q].$$

The graph of f is the **lower envelope** of the n lines. The output above is a description of this envelope on $Q \geq 0$.

3. Related work

Operations management. Locational break-even analysis is a standard topic in the location chapter of operations management textbooks, alongside the factor-rating method, the center-of-gravity method, and the transportation model (Heizer et al., 2020). The textbook treatment relies on the graph and on pairwise break-even calculations, without a general selection procedure.

Parametric linear programming. Choosing the best solution as a single parameter varies is the subject of parametric programming. Gass and Saaty (1955a) published the computational algorithm for a linear program whose objective depends on one

parameter, and Gass and Saaty (1955b) generalized the approach. In their method, the optimal solution changes at a finite set of breakpoints. Locational break-even analysis is the simplest case of this idea: each alternative is a candidate solution, Q is the parameter, and the break-even quantities are the breakpoints.

Computational geometry. The minimum of n linear functions is the lower envelope of n lines. Through point-line duality, the lower envelope of lines corresponds to a convex hull of points (Brown, 1979; de Berg et al., 2008). Planar convex hulls take $O(n \log n)$ time with Graham's scan (Graham, 1972) or with Andrew's monotone chain (Andrew, 1979). The algorithm in Section 4 is a monotone chain scan applied to lines sorted by slope.

The contribution of this paper is not a new algorithm. The contribution is a bridge: a statement of a standard operations management exercise in the language of lower envelopes, a step-by-step rule suitable for hand calculation, and a verified, open-source teaching tool.

4. Method

4.1 Two properties

Property 1 (dominance). If $VC_k \leq VC_j$ and $FC_k \leq FC_j$, then $TC_k(Q) \leq TC_j(Q)$ for all $Q \geq 0$. Alternative j never wins (or ties at best) and drops out.

Property 2 (order of winners). The function $f(Q)$ is the minimum of linear functions, so f is concave and piecewise linear. The slope of f does not increase as Q grows. The slope of f on each piece equals the VC of the winning alternative. Therefore, the winners appear in order of **decreasing VC** as volume increases. Low volume favors low fixed cost and high variable cost. High volume favors the reverse.

Property 2 justifies sorting the alternatives by VC , highest first, and scanning them once.

4.2 The three-line test

Take three alternatives i, j, k with $VC_i > VC_j > VC_k$. The middle alternative j wins on an interval of positive length if and only if

$$Q^*(i, j) < Q^*(i, k).$$

In words: j must overtake i before k does. If k overtakes i first (or at the same volume), then at every volume either i or k is at least as cheap as j , and j never wins. The three pairwise crossings of three lines are always ordered in the same way, so the test is equivalent to $Q^*(i, j) < Q^*(j, k)$.

4.3 Algorithm

```

INPUT:  alternatives (name, FC, VC)
OUTPUT: winners in order of volume; break-even quantities Q*

1. Sort alternatives by VC from highest to lowest.
   For equal VC, put the lower FC first.
2. stack ← empty
3. for each alternative k in sorted order:
   a. if stack not empty and VC(k) = VC(top): skip k
   b. while stack not empty and FC(k) ≤ FC(top):
       pop top                                     # Property 1
   c. while stack holds 2 or more lines:
       j ← top, i ← line below j
       if Q*(i, k) ≤ Q*(i, j): pop j               # three-line test
       else: break
   d. push k
4. winners ← stack, from bottom to top
5. Q*_t ← Q*(winner_t, winner_{t+1}) for each consecutive pair

```

Step 3a removes ties in VC: after sorting, a later alternative with equal VC has an equal or higher FC. Step 3b removes dominated alternatives, including alternatives whose winning interval lies only at negative volume. Step 3c applies the three-line test to the top of the stack.

4.4 Correctness sketch

By Property 2, the envelope visits winners in decreasing VC. Processing alternatives in that order, the stack holds the lower envelope of the alternatives seen so far, restricted to $Q \geq 0$, as an invariant. When k arrives, k has the lowest VC seen so far, so k wins for large enough Q . Any stack line that loses to k at every volume where the line used to win must leave the envelope. Step 3b removes lines beaten by k at $Q = 0$ and beyond. Step 3c removes the top line j when k overtakes i no later than j does, by the three-line test. Once the test fails for the top pair, lines deeper in the stack keep their intervals, because their break-even quantities with their neighbors come earlier. The invariant holds after the push. After the last alternative, the stack is the full envelope.

4.5 Complexity

Sorting takes $O(n \log n)$. Each alternative enters the stack once and leaves at most once, so the scan takes $O(n)$. Total: **$O(n \log n)$** .

For comparison, the method of computing all pairwise break-evens and testing one volume inside each interval takes $O(n^2)$ break-evens and $O(n)$ evaluations per interval, for $O(n^3)$ total. The interval method is still useful for hand calculation with three to five alternatives, and the web tool reports the stack method for any n .

5. Dual view: the convex hull of (VC, FC)

Map each alternative to the point $P_i = (VC_i, FC_i)$. The slope of the segment between P_i and P_j equals

$$(FC_j - FC_i) / (VC_j - VC_i) = -Q^*(i, j).$$

For a fixed volume Q , minimizing $FC + VC \times Q$ over the points means pushing a line of slope $-Q$ up from below until the line touches a point. The first point touched is the cheapest alternative at that volume. As Q goes from 0 to infinity, the touching points trace the **lower-left chain of the convex hull** of the points. Two consequences follow:

1. An alternative wins over some range if and only if the point lies on this chain.
2. Each break-even quantity equals minus the slope of a hull edge.

This view gives students a second picture: four points instead of four lines, with break-evens read as slopes. The stack algorithm in Section 4 is the monotone chain hull algorithm of Andrew (1979) applied to the points sorted by VC.

6. Worked example

The following textbook-style exercise appears in the author's operations management course. Fall-Line, Inc., a ski manufacturer, considers four plant locations:

Location	Annual fixed cost FC (\$)	Variable cost VC (\$ per pair)
Aspen	8,000,000	250
Boyne City	2,400,000	130
Portland	3,400,000	90
Lake Tahoe	4,500,000	65

Algorithm trace.

Step	Action	Reason	Stack after step
1	Sort by VC	Aspen (250), Boyne City (130), Portland (90), Lake Tahoe (65)	[]
2	Push Aspen	Stack empty	[Aspen]
3	Pop Aspen	Boyne City has lower VC and $FC\ 2,400,000 \leq 8,000,000$	[]
4	Push Boyne City		[Boyne City]
5	Push Portland	$FC\ 3,400,000 > 2,400,000$; one line on stack	[Boyne City, Portland]
6	Keep Portland	$Q^*(\text{Boyne City, Lake Tahoe}) = 32,308 > Q^*(\text{Boyne City, Portland}) = 25,000$	[Boyne City, Portland]
7	Push Lake Tahoe		[Boyne City, Portland, Lake Tahoe]

Break-even quantities.

$Q^*(\text{Boyne City, Portland}) = (3,400,000 - 2,400,000) / (130 - 90) = 25,000$ pairs

$Q^*(\text{Portland, Lake Tahoe}) = (4,500,000 - 3,400,000) / (90 - 65) = 44,000$ pairs

Result.

Volume range (pairs per year)	Lowest-cost location
0 to 25,000	Boyne City
25,000 to 44,000	Portland
Above 44,000	Lake Tahoe
Never	Aspen

At $Q = 25,000$, Boyne City and Portland both cost \$5,650,000. At $Q = 44,000$, Portland and Lake Tahoe both cost \$7,360,000. The crossing of Boyne City and Lake Tahoe at 32,308 pairs does not define a range, since Portland is cheaper than both at that volume (\$6,307,700 against \$6,600,000).

Hull view. The points are Aspen (250, 8.0M), Boyne City (130, 2.4M), Portland (90, 3.4M), and Lake Tahoe (65, 4.5M). The lower-left hull chain runs Boyne City → Portland → Lake Tahoe, with edge slopes $-25,000$ and $-44,000$. Aspen lies above the chain.

7. Verification

The Python implementation (`break_even_envelope.py`) includes a brute force check: for each reported range, the check evaluates every alternative at the midpoint of the range and confirms the reported winner has the lowest cost. The test suite (`test_break_even.py`) runs:

1. The worked example, asserting winners Boyne City, Portland, Lake Tahoe and break-evens 25,000 and 44,000.
2. 5,000 random instances with 1 to 10 alternatives and integer FC and VC between 0 and 100, drawn with a fixed seed (42). Small integer ranges produce frequent ties in FC, ties in VC, dominated alternatives, and three-line collinear cases, which stress steps 3a to 3c.

All tests pass. The web application implements the same steps in JavaScript and shows the trace for each input.

8. Software

The repository contains:

File	Content
index.html	Single-file web application. Runs in any browser with no installation.
break_even_envelope.py	Python implementation with a command line interface and CSV input.
test_break_even.py	Test suite against brute force.
example.csv	Data for the worked example.

Features of the web application:

1. Data entry for any number of alternatives, with currency and volume unit labels.
2. A chart of all total cost lines, with the lower envelope drawn thick, each Q* marked, and a band showing the winning range of each alternative.
3. A table of ranges and break-even quantities, and a list of alternatives never cheapest.
4. The algorithm trace, step by step, in the format of Section 6.
5. A hover readout of all costs at any volume.
6. An English and Arabic interface with right-to-left layout for Arabic. The URL parameter ?lang=ar opens the Arabic version.

Python usage:

```
python break_even_envelope.py # worked example
python break_even_envelope.py example.csv --unit pairs
python test_break_even.py
```

9. Classroom use, limitations, and extensions

Classroom use. A suggested sequence for a 50-minute session:

1. Solve the example by graph, as in the textbook.
2. Ask students how many crossings exist (six for four alternatives) and how many matter (two).
3. Teach Property 1 and remove Aspen.
4. Teach Property 2 and the three-line test. Students run the stack trace by hand.
5. Show the convex hull view as a second picture.
6. Use the web application to check answers and to explore new data, for example by lowering Aspen's fixed cost until Aspen enters the envelope.

Limitations. The model assumes linear costs, a single product, known annual volume, and no capacity limit. The analysis compares costs only. Qualitative factors such as labor skills, infrastructure, and proximity to markets need separate treatment, for example through the factor-rating method.

Extensions.

1. *Profit instead of cost.* With revenue $R \times Q$ common to all alternatives, the ranking does not change. With location-specific prices, profit lines replace cost lines and the algorithm finds the **upper** envelope by reversing the sort and the comparisons.
 2. *Capacity limits and step costs.* A site with capacity limits or a fixed-cost step creates a piecewise linear cost function. Splitting each function into segments and computing the lower envelope of segments handles this case at higher cost (de Berg et al., 2008).
 3. *Uncertain volume.* Total cost is linear in Q , so expected total cost equals TC at the expected volume. A risk-neutral decision maker uses the same ranges with Q replaced by $E[Q]$.
 4. *Multiple parameters.* When two parameters vary, for example volume and a cost escalation factor, the problem becomes a parametric program in two dimensions, the setting of Gass and Saaty (1955b).
-

10. Conclusion

Locational break-even analysis is the computation of a lower envelope of lines. Two properties, dominance and the decreasing order of variable cost, together with one three-line test, give a hand-computable rule that answers which crossings matter without a graph. The rule runs in $O(n \log n)$ time, matches brute force on 5,000 random instances, and connects an introductory operations management topic to parametric programming and computational geometry. The accompanying bilingual web application lets students in Arabic-speaking and English-speaking classrooms check their work and see each step.

References

- Andrew, A. M. (1979). Another efficient algorithm for convex hulls in two dimensions. *Information Processing Letters*, 9(5), 216–219. [https://doi.org/10.1016/0020-0190\(79\)90072-3](https://doi.org/10.1016/0020-0190(79)90072-3)
- Assaf, R. (2026). *Break-even location finder* (Version 1.0) [Computer software]. GitHub. <https://github.com/ramizassaf/break-even-location-finder>
- Brown, K. Q. (1979). Voronoi diagrams from convex hulls. *Information Processing Letters*, 9(5), 223–228. [https://doi.org/10.1016/0020-0190\(79\)90074-7](https://doi.org/10.1016/0020-0190(79)90074-7)
- de Berg, M., Cheong, O., van Kreveld, M., & Overmars, M. (2008). *Computational geometry: Algorithms and applications* (3rd ed.). Springer.
- Gass, S., & Saaty, T. (1955a). The computational algorithm for the parametric objective function. *Naval Research Logistics Quarterly*, 2(1–2), 39–45. <https://doi.org/10.1002/nav.3800020106>
- Gass, S. I., & Saaty, T. L. (1955b). Parametric objective function (Part 2): Generalization. *Operations Research*, 3(4), 395–401.

Graham, R. L. (1972). An efficient algorithm for determining the convex hull of a finite planar set. *Information Processing Letters*, 1(4), 132–133.

Heizer, J., Render, B., & Munson, C. (2020). *Operations management: Sustainability and supply chain management* (13th ed.). Pearson.

How to cite this white paper

Assaf, R. (2026). *Locational break-even analysis without plotting: A lower-envelope algorithm and an open-source bilingual teaching tool* (White paper, Version 1.0). An-Najah National University. <https://github.com/ramizassaf/break-even-location-finder>

BibTeX:

```
@techreport{assaf2026breakeven,  
  author      = {Assaf, Ramiz},  
  title       = {Locational Break-Even Analysis Without Plotting: A Lower-Envelope  
                Algorithm and an Open-Source Bilingual Teaching Tool},  
  institution = {An-Najah National University},  
  type        = {White paper},  
  number      = {Version 1.0},  
  year        = {2026},  
  url         = {https://github.com/ramizassaf/break-even-location-finder}  
}
```